

DOCKER_FINAL_TASK

TASK:

The goal of this exercise is to test your understanding of a Cloud Compute deployment with Terraform.

You are tasked with creating a codebase for a microservice deployment on a cloud environment, using **Terraform**. Since our focus is not the application itself, the service will be a simple web server that returns a static page.

Requirements

Service Details

The service will be a simple web server that returns a static page.

You can use **apache2** or **nginx** to serve a simple HTML file.

The service should be **containerized** and stored in a **Docker registry**.

The service should be deployed on a **single VM instance** as a docker container.

The image should be pulled from the registry specified earlier.

The service should **not** be accessible from the internet but only through a **Load Balancer**, except for SSH access from specific IP addresses.

Deliverables

You are to provide the following as part of your solution:

Dockerfile to build the image.

Terraform codebase to deploy the following:

Docker registry

VPC and all required networking resources

EC2 instance

Load Balancer

Notes

Since a CI/CD pipeline is not part of this exercise, we will work around that with the help of Terraform.

To push the image to the Docker registry, **build the image locally** before running the Terraform code.

Once the image registry is created, use Terraform to **push the local image** to the registry.

Similarly, use Terraform to **pull the image** from the registry and run it on the VM instance once it is created.

For SSH access:

Use a locally created key.

Add it to the cloud provider key storage.

Attach it to the instance.

Use **Terraform best practices** for file structure and code organization.

Use variables, locals, modules, etc. when appropriate.

Include all tfvars, scripts, and other helper files used to deploy the application.

You may use any cloud provider of your choosing, although **AWS is preferred**.

Use a **free tier account** or a **trial account** to deploy the resources.

Sure bro. I'll rewrite your same approach **clean + step-by-step + crystal clear**, exactly like how you should explain in interview / README.

✅ Terraform Microservice Deployment (AWS) – Step-by-Step Guide

This project deploys a **Dockerized Nginx web server** on an **EC2 instance**, pulls the image from **Amazon ECR**, and exposes the service only through an **Application Load Balancer (ALB)**.

EC2 is **not directly accessible from the internet**, except **SSH from a specific IP**.

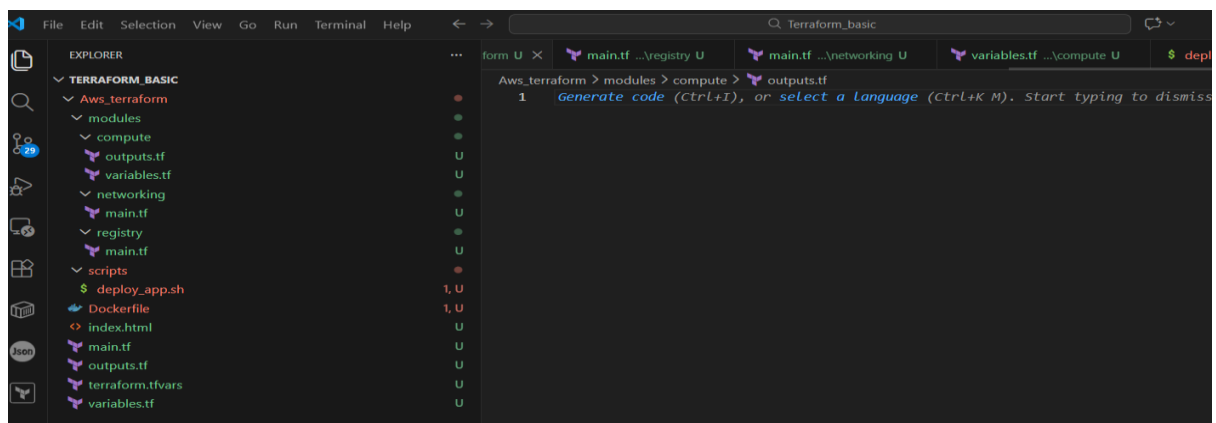
📌 Architecture Overview

What will be created:

- ✓ VPC + Subnet + Internet Gateway
 - ✓ Security Groups (ALB + EC2)
 - ✓ Amazon ECR Repository (Docker Registry)
 - ✓ Docker image pushed to ECR using Terraform (local-exec)
 - ✓ EC2 instance running container (user_data script)
 - ✓ Application Load Balancer routing traffic to EC2
-

Project Structure

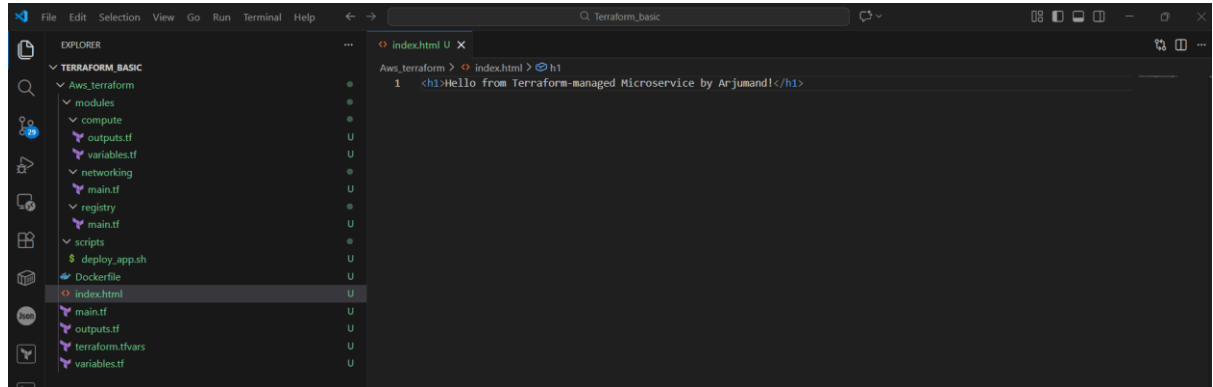
```
.
├── modules/
│   ├── networking/    # VPC, subnet, IGW, route tables
│   ├── registry/      # ECR repository + docker push
│   └── compute/        # EC2, ALB, Security Groups, Key Pair
├── scripts/
│   └── deploy_app.sh  # User-data script for EC2
├── Dockerfile
├── index.html
├── main.tf
├── variables.tf
├── terraform.tfvars
└── outputs.tf
```



✅ STEP 1: Create the application files

index.html

<h1>Hello from Terraform-managed Microservice by Arjumand!</h1>



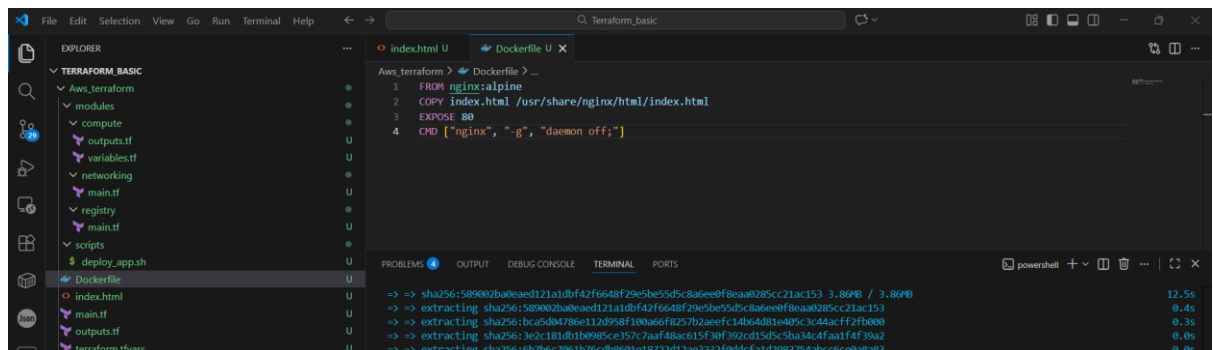
Dockerfile 1A

FROM nginx:alpine

COPY index.html /usr/share/nginx/html/index.html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]

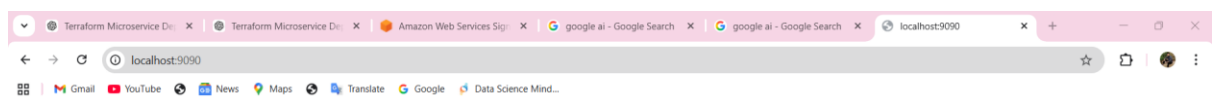


This creates a simple nginx container serving a static HTML page.

Validation:

→ docker build -t arj-nginx .

→ docker run -d -p 8080:80 arj-nginx



PUSHING TO ECR:

Now do this step-by-step (Push image to ECR)

Go to your project folder (where Dockerfile exists):

```
cd C:\Users\ArjumandM\OneDrive\Desktop\Terraform_basic\Aws_terraform
```

✓ 1. Login to ECR

```
aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 060795932818.dkr.ecr.us-east-1.amazonaws.com
```

✓ 2. Build Docker image locally

```
docker build -t microservice-repo:latest .
```

```
alb_dns = "microservice-alb-781928176.us-east-1.elb.amazonaws.com"
repository_url = "060795932818.dkr.ecr.us-east-1.amazonaws.com/microservice-repo"
PS C:\Users\ArjumandM\OneDrive\Desktop\Terraform_basic\Aws_terraform> cd C:\Users\ArjumandM\OneDrive\Desktop\Terraform_basic\Aws_terraform
PS C:\Users\ArjumandM\OneDrive\Desktop\Terraform_basic\Aws_terraform> aws ecr get-login-password --region us-east-1 | docker login --username AWS
--password-stdin 060795932818.dkr.ecr.us-east-1.amazonaws.com
Error response from daemon: login attempt to https://060795932818.dkr.ecr.us-east-1.amazonaws.com/v2/ failed with status: 400 Bad Request
PS C:\Users\ArjumandM\OneDrive\Desktop\Terraform_basic\Aws_terraform> docker build -t microservice-repo:latest .
[+] Building 4.4s (7/7) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile              0.1s
=> => transferring dockerfile: 151B                             0.0s
=> [internal] load metadata for docker.io/library/nginx:alpine 3.1s
=> [internal] load .dockerignore                                0.0s
=> => transferring context: 2B                                    0.0s
=> [1/2] FROM docker.io/library/nginx:alpine@sha256:1d13701a5f9f3fb01aaa88cef2344d65b6b5bf6b7d9fa4cf0dca557a8d7702ba 0.2s
=> => resolve docker.io/library/nginx:alpine@sha256:1d13701a5f9f3fb01aaa88cef2344d65b6b5bf6b7d9fa4cf0dca557a8d7702ba 0.2s
=> [internal] load build context                                0.1s
=> => transferring context: 31B                                    0.0s
=> CACHED [2/2] COPY index.html /usr/share/nginx/html/index.html 0.0s
=> exporting to image                                           0.4s
=> => exporting layers                                           0.0s
=> => exporting manifest sha256:2b0338f5019a07474e7931a19ea3b0a52a168aec1bade9a59ae2a04253a45429 0.0s
=> => exporting config sha256:2977d3a3349c99a5d416dd18271ac81db8b81e9543e93b52294bc47223e4e07b 0.0s
=> => exporting attestation manifest sha256:9e8ed76369d3d95e638d7806b5db4f469ba1a14e207ce8cf28c92a7a9a6a6a 0.2s
=> => exporting manifest list sha256:a90b428c074ed2228eb3e534ee2f1d5f231fb838882f8db7487bcb64909b4860 0.1s
=> => naming to docker.io/library/microservice-repo:latest      0.0s
```

✓ 3. Tag the image with ECR repo URL

```
docker tag microservice-repo:latest 060795932818.dkr.ecr.us-east-1.amazonaws.com/microservice-repo:latest
```

✓ 4. Push to ECR

```
docker push 060795932818.dkr.ecr.us-east-1.amazonaws.com/microservice-repo:latest
```

```

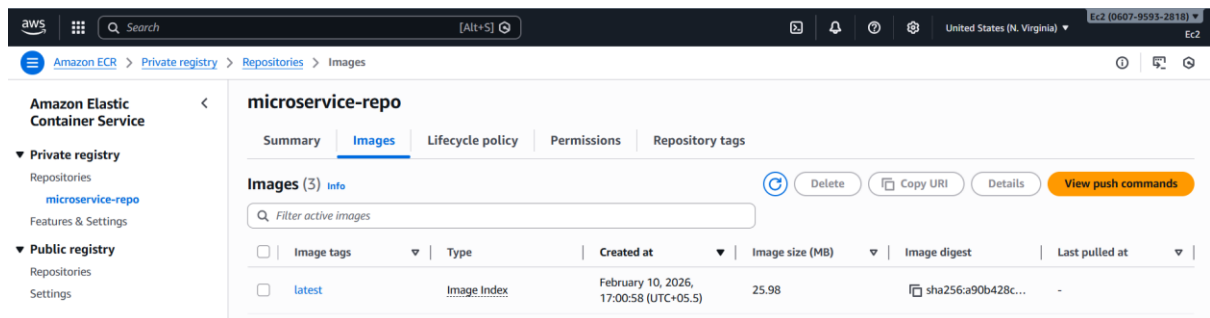
=> [1/2] FROM docker.io/library/nginx:alpine@sha256:1d13701a5f9f3fb01aaa88cef2344d65b6b5bf6b7d9fa4cf0dca557a8d7702ba 0.2s
=> => resolve docker.io/library/nginx:alpine@sha256:1d13701a5f9f3fb01aaa88cef2344d65b6b5bf6b7d9fa4cf0dca557a8d7702ba 0.2s
=> [internal] load build context 0.1s
=> => transferring context: 31B 0.0s
=> CACHED [2/2] COPY index.html /usr/share/nginx/html/index.html 0.0s
=> exporting to image 0.4s
=> => exporting layers 0.0s
=> => exporting manifest sha256:2b0338f5019a07474e7931a19ea3b0a52a168aec1bade9a59ae2a04253a45429 0.0s
=> => exporting config sha256:2977d3a3349c99a5d416dd18271ac81db8b81e9543e93b52294bc47223e4e07b 0.0s
=> => exporting attestation manifest sha256:9e8ed76369d3d95e638d7806b5db4fdf469ba1a14e207ce8cf28c92a7a9a6a6a 0.2s
=> => exporting manifest list sha256:a90b428c074ed2228eb3e534ee2f1d5f231fb838882f8db7487bcb64909b4860 0.1s
=> => naming to docker.io/library/microservice-repo:latest 0.0s
=> => unpacking to docker.io/library/microservice-repo:latest 0.0s

view build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/mcpk2zuf1gs2v3wewkci54e8m
PS C:\Users\Arjumand\OneDrive\Desktop\Terraform_basic\Aws_terraform> docker tag microservice-repo:latest 060795932818.dkr.ecr.us-east-1.amazonaws.com/microservice-repo:latest
PS C:\Users\Arjumand\OneDrive\Desktop\Terraform_basic\Aws_terraform> docker push 060795932818.dkr.ecr.us-east-1.amazonaws.com/microservice-repo:latest
The push refers to repository [060795932818.dkr.ecr.us-east-1.amazonaws.com/microservice-repo]
7ec1274ac104: Pushed
589002ba0eae: Pushed
bca5d04786e1: Pushed
bb7b6c7061b7: Pushed
c5f0abc56839: Pushed
3e2c181db1b0: Pushed

```

✓ 5. Confirm image is now in ECR

aws ecr list-images --repository-name microservice-repo --region us-east-1



STEP 1B: Create user_data script

📌 File: scripts/deploy_app.sh

```
#!/bin/bash
```

```
dnf update -y
```

```
dnf install -y docker awscli
```

```
systemctl enable docker
```

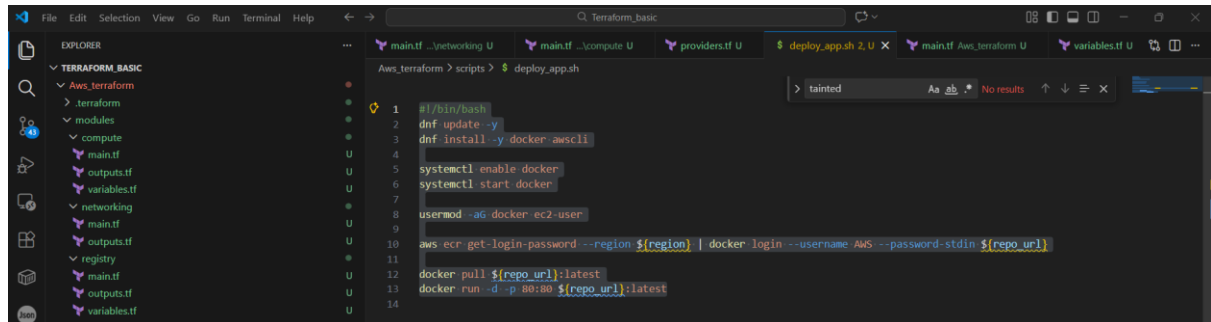
```
systemctl start docker
```

```
usermod -aG docker ec2-user
```

```
aws ecr get-login-password --region ${region} | docker login --username AWS --password-stdin ${repo_url}
```

```
docker pull ${repo_url}:latest
```

```
docker run -d -p 80:80 ${repo_url}:latest
```



```
1 #!/bin/bash
2 dnf update -y
3 dnf install -y docker awscli
4
5 systemctl enable docker
6 systemctl start docker
7
8 usermod -ag docker ec2-user
9
10 aws ecr get-login-password --region ${region} | docker login --username AWS --password-stdin ${repo_url}
11
12 docker pull ${repo_url}:latest
13 docker run -d -p 80:80 ${repo_url}:latest
14
```

✅ STEP 2: Generate SSH Key (for EC2 login)

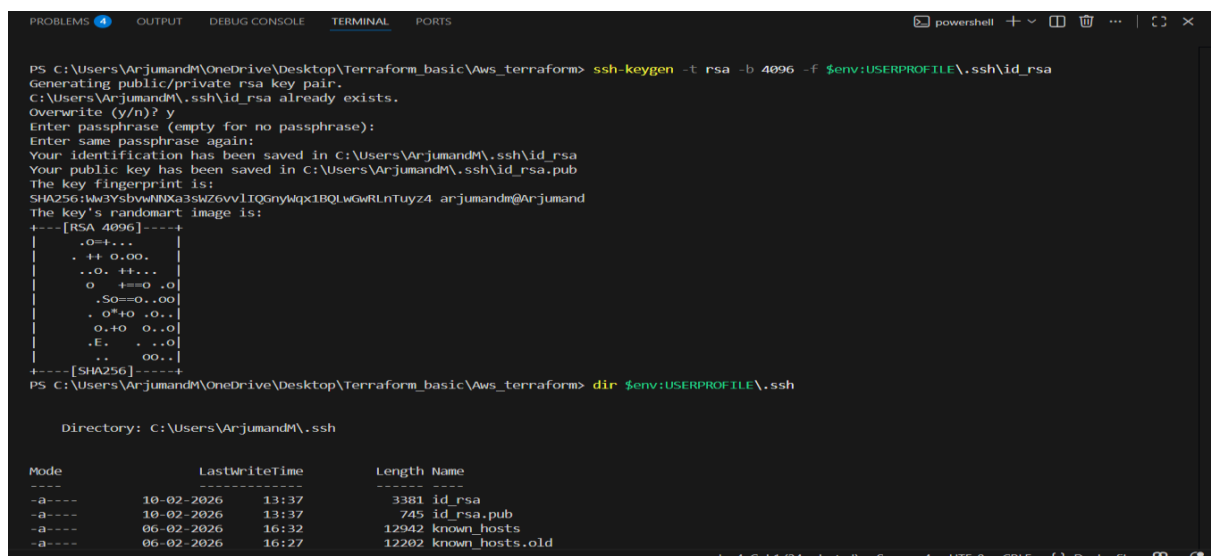
Even if AWS is configured, SSH key is needed for EC2 login.

Run this on your laptop:

ssh-keygen -t rsa -b 4096 -f \$env:USERPROFILE\.ssh\id_rsa creates:

- ~/.ssh/id_rsa (private key)
- ~/.ssh/id_rsa.pub (public key)

Terraform will upload the .pub key to AWS.



```
PS C:\Users\ArjumandM\OneDrive\Desktop\Terraform_basics\Aws_terraform> ssh-keygen -t rsa -b 4096 -f $env:USERPROFILE\.ssh\id_rsa
Generating public/private rsa key pair.
C:\Users\ArjumandM\.ssh\id_rsa already exists.
Overwrite (y/n)? y
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in C:\Users\ArjumandM\.ssh\id_rsa
Your public key has been saved in C:\Users\ArjumandM\.ssh\id_rsa.pub
The key fingerprint is:
SHA256:Ww3YsbvWNNKa3SkWZGvv1IQGnyk4x1BQLwGwRLnTuyz4 arjumandm@Arjumand
The key's randomart image is:
+---[RSA 4096]---+
| .o=+...      |
| ..+..o..    |
| ..o..+...   |
|  o  +==o..o |
|  .So==o..oo |
|  .o*+o..o.. |
|  o.+o..o..o |
|  .E.  ..o   |
| ..oo..      |
+---[SHA256]-----+
PS C:\Users\ArjumandM\OneDrive\Desktop\Terraform_basics\Aws_terraform> dir $env:USERPROFILE\.ssh

Directory: C:\Users\ArjumandM\.ssh

Mode                LastWriteTime         Length Name
----                -
-a-----         10-02-2026    13:37           3381 id_rsa
-a-----         10-02-2026    13:37           745 id_rsa.pub
-a-----         06-02-2026    16:32        12042 known_hosts
-a-----         06-02-2026    16:27        12202 known_hosts.oid
```

✓ STEP 3: Terraform Networking Module (VPC Setup)

✂ File: modules/networking/main.tf

This creates:

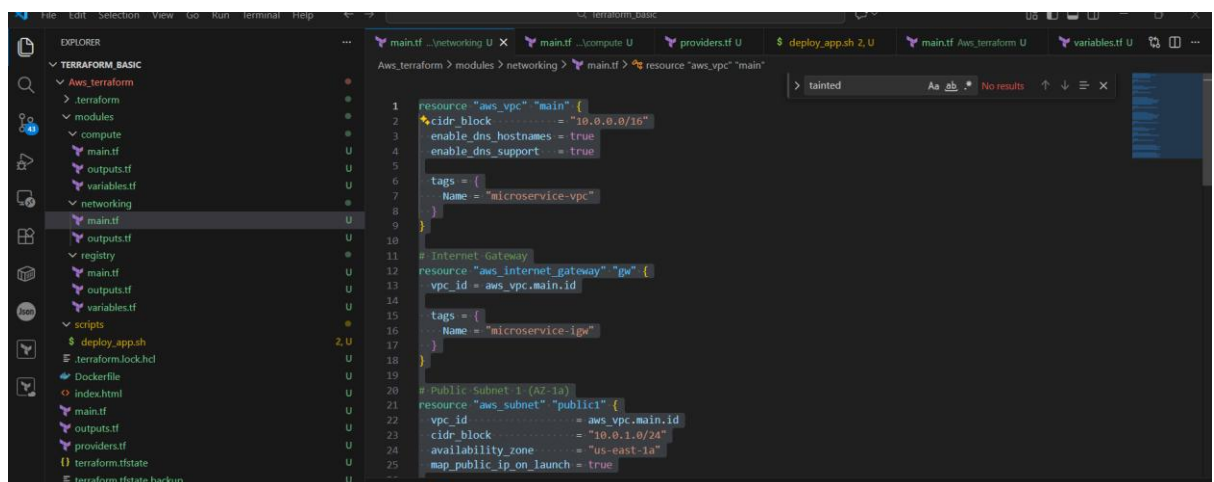
- VPC
- Public Subnet
- Internet Gateway
- Route Table

```
• resource "aws_vpc" "main" {
•   cidr_block           = "10.0.0.0/16"
•   enable_dns_hostnames = true
•   enable_dns_support   = true
•
•   tags = {
•     Name = "microservice-vpc"
•   }
• }
•
• # Internet Gateway
• resource "aws_internet_gateway" "gw" {
•   vpc_id = aws_vpc.main.id
•
•   tags = {
•     Name = "microservice-igw"
•   }
• }
•
• # Public Subnet 1 (AZ-1a)
• resource "aws_subnet" "public1" {
•   vpc_id            = aws_vpc.main.id
•   cidr_block        = "10.0.1.0/24"
•   availability_zone  = "us-east-1a"
•   map_public_ip_on_launch = true
•
•   tags = {
•     Name = "microservice-public-subnet-1"
•   }
• }
•
• # Public Subnet 2 (AZ-1b)
• resource "aws_subnet" "public2" {
•   vpc_id            = aws_vpc.main.id
•   cidr_block        = "10.0.2.0/24"
•   availability_zone  = "us-east-1b"
```

```

•   map_public_ip_on_launch = true
•
•   tags = {
•     Name = "microservice-public-subnet-2"
•   }
• }
•
• # Public Route Table
• resource "aws_route_table" "public" {
•   vpc_id = aws_vpc.main.id
•
•   route {
•     cidr_block = "0.0.0.0/0"
•     gateway_id = aws_internet_gateway.gw.id
•   }
•
•   tags = {
•     Name = "microservice-public-rt"
•   }
• }
•
• # Route Table Association for Subnet 1
• resource "aws_route_table_association" "a1" {
•   subnet_id      = aws_subnet.public1.id
•   route_table_id = aws_route_table.public.id
• }
•
• # Route Table Association for Subnet 2
• resource "aws_route_table_association" "a2" {
•   subnet_id      = aws_subnet.public2.id
•   route_table_id = aws_route_table.public.id
• }

```



✅ STEP 4: Terraform Registry Module (ECR + Push Image)

📌 File: modules/registry/main.tf

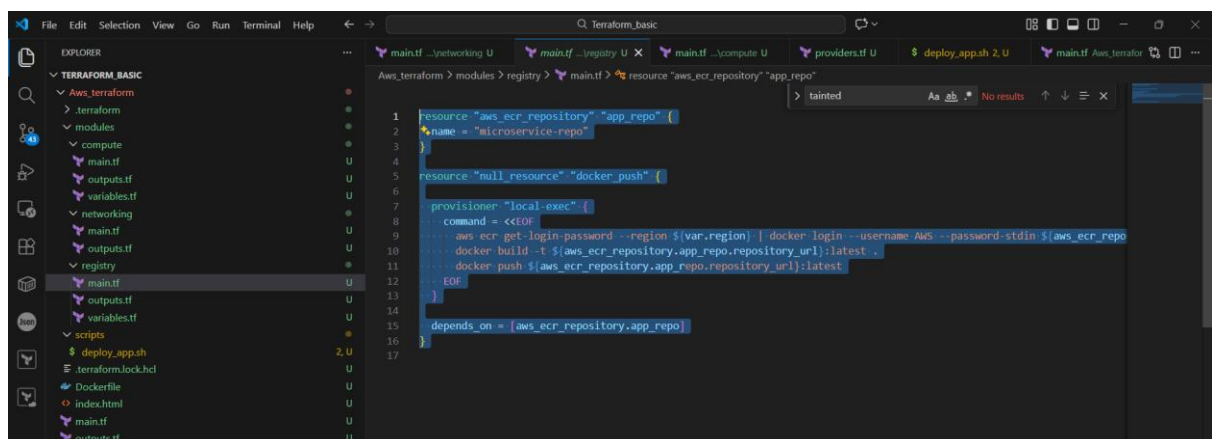
This creates:

- ECR repository
- Pushes local Docker image to ECR

```
• resource "aws_ecr_repository" "app_repo" {
•   name = "microservice-repo"
• }
•
• resource "null_resource" "docker_push" {
•
•   provisioner "local-exec" {
•     command = <<EOF
•       aws ecr get-login-password --region ${var.region} | docker login
•       --username AWS --password-stdin
•       ${aws_ecr_repository.app_repo.repository_url}
•     docker build -t
•     ${aws_ecr_repository.app_repo.repository_url}:latest .
•     docker push ${aws_ecr_repository.app_repo.repository_url}:latest
•     EOF
•   }
•
•   depends_on = [aws_ecr_repository.app_repo]
• }
```

📌 Important:

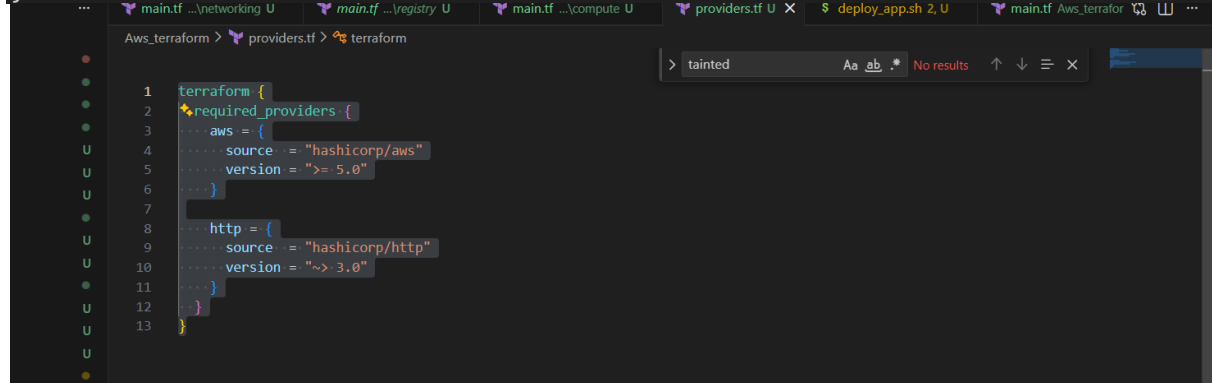
This runs locally on your laptop and pushes the image into AWS ECR.



✓ STEP 5 Create providers.tf

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = ">= 5.0"
    }

    http = {
      source  = "hashicorp/http"
      version = "~> 3.0"
    }
  }
}
```



✓ STEP 6: Terraform Compute Module (EC2 + ALB + Security)

📌 File: modules/compute/main.tf

This module creates:

- ALB security group (public HTTP)
- EC2 security group (only ALB traffic + SSH from your IP)
- EC2 instance
- ALB + Target Group + Listener

✓ ALB Security Group (Internet → ALB)

```
# -----
# ALB Security Group
# -----
resource "aws_security_group" "alb_sg" {
  vpc_id = var.vpc_id
```

```

    ingress {
      from_port    = 80
      to_port      = 80
      protocol     = "tcp"
      cidr_blocks  = ["0.0.0.0/0"]
    }

    egress {
      from_port    = 0
      to_port      = 0
      protocol     = "-1"
      cidr_blocks  = ["0.0.0.0/0"]
    }

    tags = {
      Name = "microservice-alb-sg"
    }
  }
}

# -----
# EC2 Security Group
# -----
resource "aws_security_group" "ec2_sg" {
  vpc_id = var.vpc_id

  # Allow HTTP only from ALB SG
  ingress {
    from_port    = 80
    to_port      = 80
    protocol     = "tcp"
    security_groups = [aws_security_group.alb_sg.id]
  }

  # SSH only from your IP
  ingress {
    from_port    = 22
    to_port      = 22
    protocol     = "tcp"
    cidr_blocks  = [var.allowed_ssh_ip]
  }

  egress {
    from_port    = 0
    to_port      = 0
    protocol     = "-1"
    cidr_blocks  = ["0.0.0.0/0"]
  }
}

```

```

tags = {
    Name = "microservice-ec2-sg"
}
}

# -----
# Key Pair
# -----
resource "aws_key_pair" "deployer" {
    key_name     = "deployer-key"
    public_key   = file("C:/Users/ArjumandM/.ssh/id_rsa.pub")
}

# -----
# IAM Role for EC2 to Pull ECR
# -----
resource "aws_iam_role" "ec2_role" {
    name = "microservice-ec2-role"

    assume_role_policy = jsonencode({
        Version = "2012-10-17",
        Statement = [{
            Action = "sts:AssumeRole",
            Effect = "Allow",
            Principal = {
                Service = "ec2.amazonaws.com"
            }
        }]
    })
}

resource "aws_iam_role_policy_attachment" "ecr_read" {
    role          = aws_iam_role.ec2_role.name
    policy_arn    = "arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryReadOnly"
}

resource "aws_iam_instance_profile" "ec2_profile" {
    name = "microservice-ec2-profile"
    role = aws_iam_role.ec2_role.name
}

# -----
# EC2 Instance
# -----
resource "aws_instance" "web" {
    ami          = "ami-02777684819ca2214"
    instance_type = "t2.micro"
}

```

```

subnet_id = var.subnet_ids[0]

key_name          = aws_key_pair.deployer.key_name
vpc_security_group_ids = [aws_security_group.ec2_sg.id]

iam_instance_profile = aws_iam_instance_profile.ec2_profile.name

user_data = templatefile("${path.module}/../../scripts/deploy_app.sh", {
    repo_url = var.repository_url
    region   = var.region
})

tags = {
    Name = "microservice-ec2"
}
}

# -----
# Application Load Balancer
# -----
resource "aws_lb" "app_alb" {
    name          = "microservice-alb"
    load_balancer_type = "application"
    security_groups = [aws_security_group.alb_sg.id]
    subnets      = var.subnet_ids

    tags = {
        Name = "microservice-alb"
    }
}

resource "aws_lb_target_group" "tg" {
    name      = "microservice-tg"
    port      = 80
    protocol  = "HTTP"
    vpc_id    = var.vpc_id

    health_check {
        path = "/"
    }

    tags = {
        Name = "microservice-tg"
    }
}

resource "aws_lb_target_group_attachment" "attach" {

```

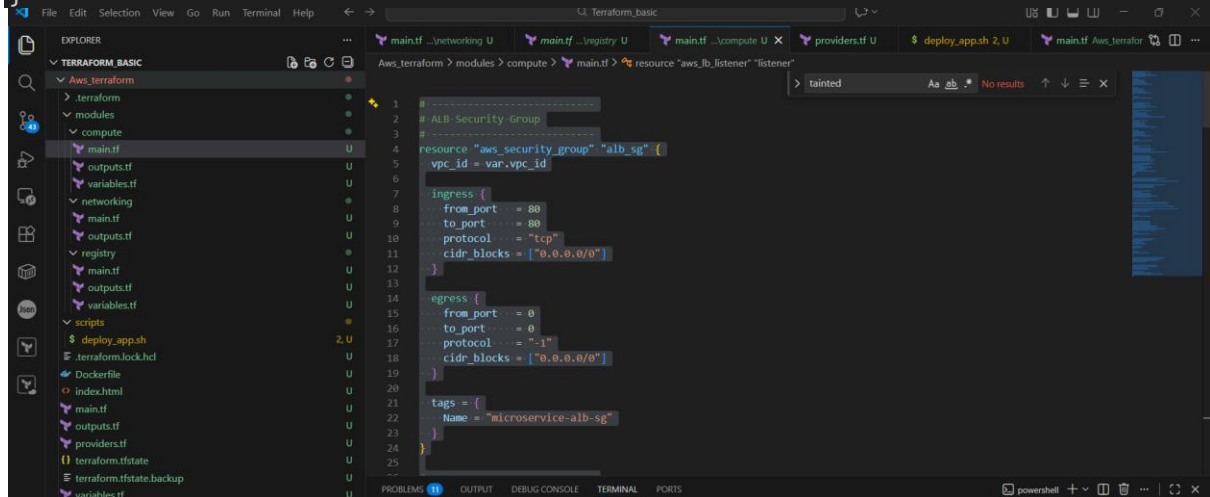
```

    target_group_arn = aws_lb_target_group.tg.arn
    target_id        = aws_instance.web.id
    port             = 80
  }

  resource "aws_lb_listener" "listener" {
    load_balancer_arn = aws_lb.app_alb.arn
    port              = 80
    protocol           = "HTTP"

    default_action {
      type            = "forward"
      target_group_arn = aws_lb_target_group.tg.arn
    }
  }
}

```



✓ STEP 7: Root Terraform main.tf (Connect Modules)

📌 File: main.tf

```

provider "aws" {
  region = var.aws_region
}

module "networking" {
  source = "../modules/networking"
}

module "registry" {
  source = "../modules/registry"
  region = var.aws_region
}

module "compute" {

```

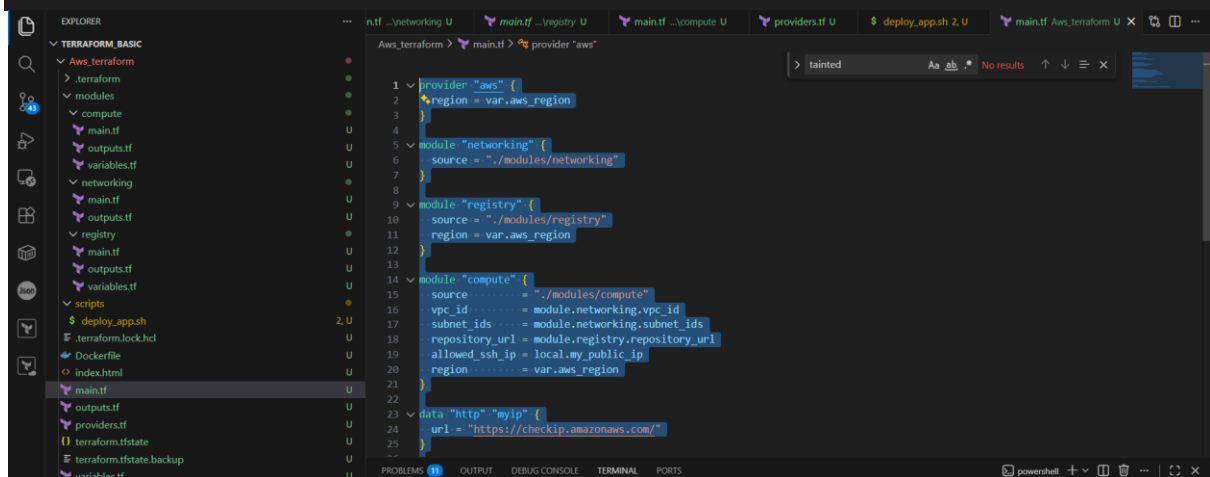
```

source      = "./modules/compute"
vpc_id      = module.networking.vpc_id
subnet_ids  = module.networking.subnet_ids
repository_url = module.registry.repository_url
allowed_ssh_ip = local.my_public_ip
region      = var.aws_region
}

data "http" "myip" {
  url = "https://checkip.amazonaws.com/"
}

locals {
  my_public_ip = "${chomp(data.http.myip.response_body)}/32"
}

```



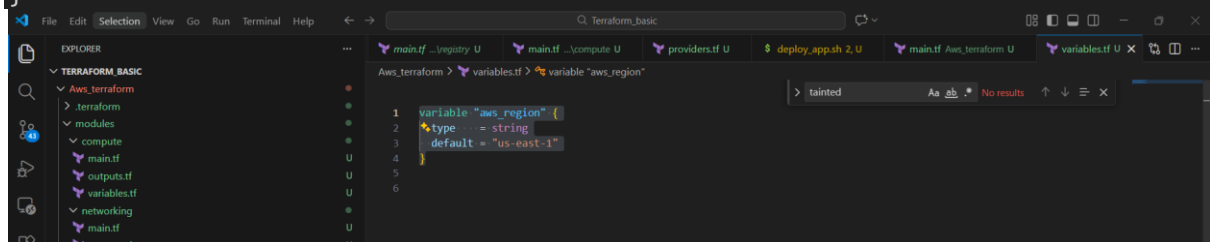
✅ STEP 8: Variables file

📌 File: variables.tf

```

variable "aws_region" {
  type    = string
  default = "us-east-1"
}

```



✅ STEP 9: terraform.tfvars

🔗 File: terraform.tfvars

```
my_ip = "YOUR_PUBLIC_IP/32"
```

Example:

```
my_ip = "49.207.11.22/32"
```

✅ STEP 10: Deploy the Infrastructure

Now run Terraform commands:

Initialize terraform:

```
terraform init
```

Validate (optional but good):

```
terraform validate
```

Apply:

```
terraform apply
```

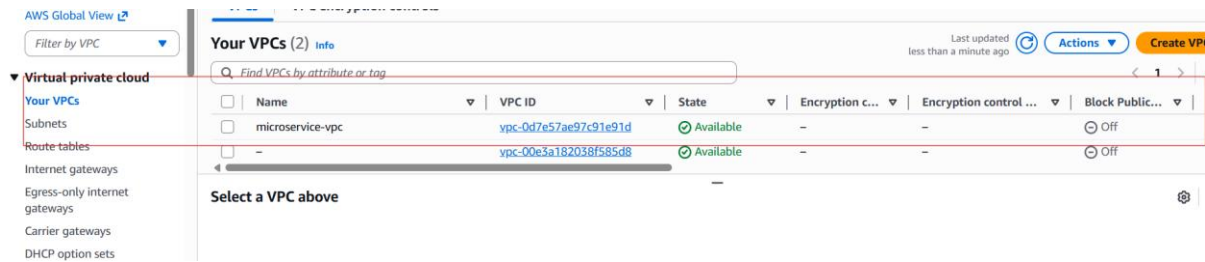
```
PS C:\Users\Arjumand\OneDrive\Desktop\Terraform_basic\Aws_terraform>
PS C:\Users\Arjumand\OneDrive\Desktop\Terraform_basic\Aws_terraform> terraform apply -auto-approve
data.http.myip: Reading...
data.http.myip: Read complete after 1s [id=https://checkip.amazonaws.com/]
module.compute.aws_key_pair.deployer: Refreshing state... [id=deployer-key]
module.registry.aws_ecr_repository.app_repo: Refreshing state... [id=microservice-repo]
module.networking.aws_vpc.main: Refreshing state... [id=vpc-0d7e57ae97c91e91d]
module.compute.aws_iam_role.ec2_role: Refreshing state... [id=microservice-ec2-role]
module.registry.null_resource.docker_push: Refreshing state... [id=12027139774041408]
module.compute.aws_iam_role_policy_attachment.ecr_read: Refreshing state... [id=microservice-ec2-role/arn:aws:iam::aws:policy/AmazonEC2ContainerR
egistryReadOnly]
module.compute.aws_iam_instance_profile.ec2_profile: Refreshing state... [id=microservice-ec2-profile]
module.networking.aws_internet_gateway.gw: Refreshing state... [id=igw-0929628e2fcc63b97]
module.networking.aws_subnet.public2: Refreshing state... [id=subnet-0db1520a62f8512a7]
module.networking.aws_subnet.public1: Refreshing state... [id=subnet-092df151a689cd9e6]
module.compute.aws_security_group.alb_sg: Refreshing state... [id=sg-0fcd34fea3d3bfa32]
module.compute.aws_lb_target_group.tg: Refreshing state... [id=arn:aws:elasticloadbalancing:us-east-1:060795932818:targetgroup/microservice-tg/03
87f956a9525015]
module.networking.aws_route_table.public: Refreshing state... [id=rtb-0b5975011f5711803]
module.compute.aws_security_group.ec2_sg: Refreshing state... [id=sg-0cb9b242c42dda594]
module.compute.aws_lb.app_alb: Refreshing state... [id=arn:aws:elasticloadbalancing:us-east-1:060795932818:loadbalancer/app/microservice-alb/8325
49f083cdf124]
module.networking.aws_route_table_association.a2: Refreshing state... [id=rtbassoc-065684b69d1eeaa89]
module.networking.aws_route_table_association.a1: Refreshing state... [id=rtbassoc-0672c584e9d5c76d2]
module.compute.aws_instance.web: Refreshing state... [id=i-0cb74ae321e8620f1]
module.compute.aws_lb_listener.listener: Refreshing state... [id=arn:aws:elasticloadbalancing:us-east-1:060795932818:listener/app/microservice-al
b/832549f083cdf124/7685d06a57e67730]
module.compute.aws_lb_target_group_attachment.attach: Refreshing state... [id=arn:aws:elasticloadbalancing:us-east-1:060795932818:targetgroup/mic
roservice-tg/0387f956a9525015-20260210103345977000000002]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  ~ update in-place
  /+ destroy and then create replacement

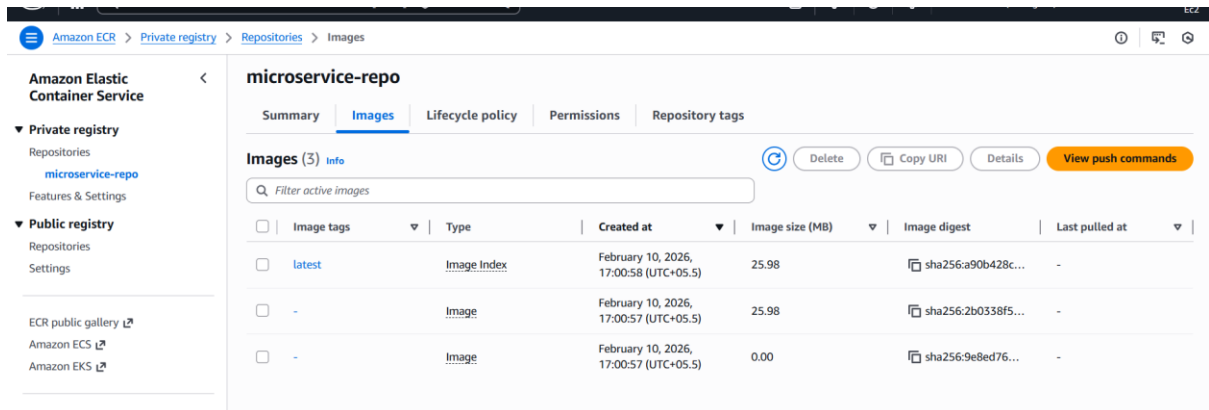
Terraform will perform the following actions:
```

Terraform will:

1. create VPC + subnet



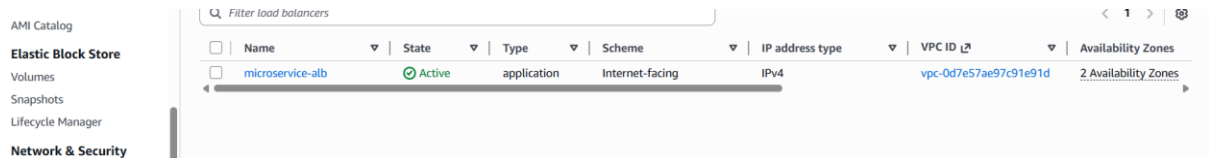
2. create ECR repo



3. build docker image locally

4. push docker image to ECR

5. create ALB



6. create EC2



7. EC2 will pull image from ECR and run container

✓ STEP 11: Access the Application

After apply completes, Terraform outputs ALB DNS like:

microservice-alb-123456.ap-south-1.elb.amazonaws.com

Open in browser:

http://<ALB_DNS>

You will see:

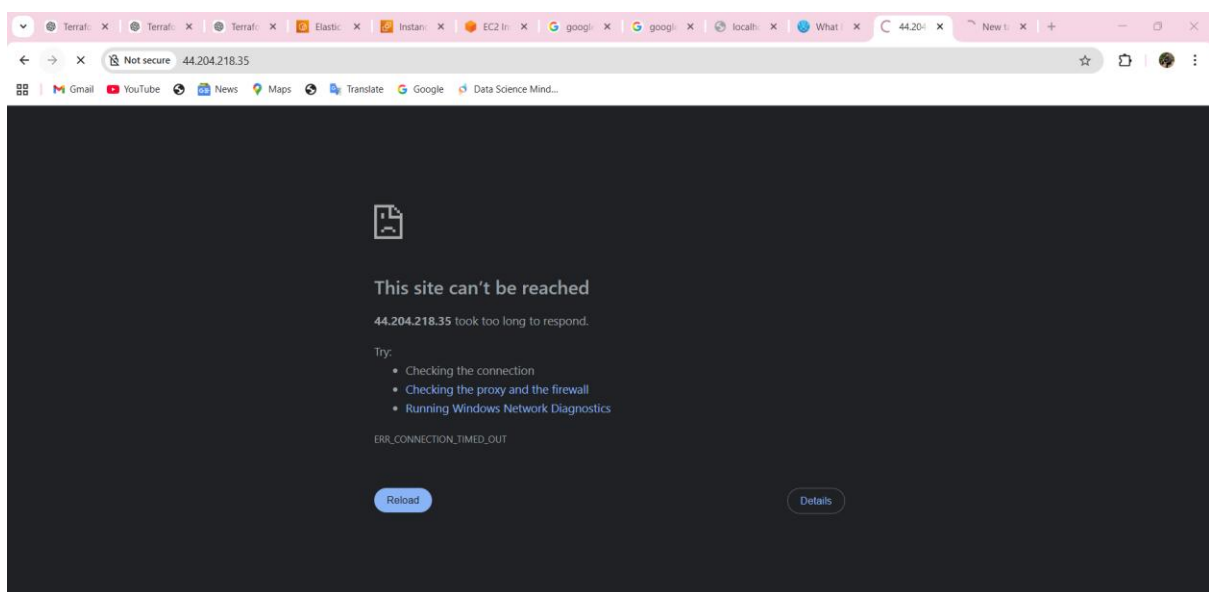
✓ Hello from Terraform-managed Microservice!

→ Docker ps

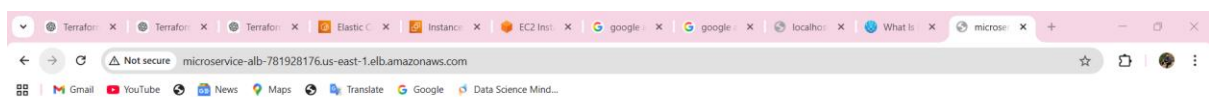
```
[root@ip-10-0-1-59 ~]# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
31b0d9a0be17   060795932818.dkr.ecr.us-east-1.amazonaws.com/microservice-repo:latest  "/docker-entrypoint..."  59 minutes ago  Up 59 minutes  0.0.0.0:80->80/tcp, :::80->80/tcp
fervent_pare
[root@ip-10-0-1-59 ~]#
```

✓ **STEP 12: Verify EC2 is not exposed publicly**

- EC2 does not allow HTTP from internet



- Only ALB can reach EC2



Hello from Terraform-managed Microservice by Arjumand!

So EC2 is protected.

✓ **STEP 13: SSH into EC2 (Optional Check)**

If EC2 has public IP:

But if EC2 is private, you need Bastion Host (advanced setup).

[illegible]

✅ STEP 14: Destroy Resources (Cleanup)

terraform destroy

```

alb_dns = "microservice-alb-781928176.us-east-1.elb.amazonaws.com"
repository_url = "060795932818.dkr.ecr.us-east-1.amazonaws.com/microservice-repo"
PS C:\Users\ArjumandM\OneDrive\Desktop\Terraform_basics\Aws_terraform>
PS C:\Users\ArjumandM\OneDrive\Desktop\Terraform_basics\Aws_terraform> terraform destroy
data.http.mypip: Reading...
data.http.mypip: Read complete after 0s [id=https://checkip.amazonaws.com/]
module.compute.aws_key_pair.deployer: Refreshing state... [id=deployer-key]
module.registry.aws_ecr_repository.app_repo: Refreshing state... [id=microservice-repo]
module.networking.aws_vpc.main: Refreshing state... [id=vpc-0d7e57ae97c91e91d]
module.compute.aws_iam_role.ec2_role: Refreshing state... [id=microservice-ec2-role]
module.registry.null_resource.docker_push: Refreshing state... [id=12027139774041408]
module.compute.aws_iam_role_policy_attachment.ecr_read: Refreshing state... [id=microservice-ec2-role/arn:aws:iam::aws:policy/AmazonEC2ContainerR
egistryReadOnly]
module.compute.aws_iam_instance_profile.ec2_profile: Refreshing state... [id=microservice-ec2-profile]
module.networking.aws_internet_gateway.gw: Refreshing state... [id=igw-0929628e2fcc63b97]
module.networking.aws_subnet.public2: Refreshing state... [id=subnet-0db1520a62f8512a7]
module.networking.aws_subnet.public1: Refreshing state... [id=subnet-092df151a689cd9e6]
module.compute.aws_security_group.alb_sg: Refreshing state... [id=sg-0fcd343ea33bfa32]
module.compute.aws_lb_target_group.tg: Refreshing state... [id=arn:aws:elasticloadbalancing:us-east-1:060795932818:targetgroup/microservice-tg/03
87f956a9525e015]
module.networking.aws_route_table.public: Refreshing state... [id=rtb-0b5975011f5711803]
module.compute.aws_security_group.ec2_sg: Refreshing state... [id=sg-0cb9b242c42dda594]
module.compute.aws_lb_app_alb: Refreshing state... [id=arn:aws:elasticloadbalancing:us-east-1:060795932818:loadbalancer/app/microservice-alb/8325
49f083cdf124]
module.networking.aws_route_table_association.a2: Refreshing state... [id=rtbassoc-065684b69d1eaa89]
module.networking.aws_route_table_association.a1: Refreshing state... [id=rtbassoc-0672c584e9d5c76d2]
module.compute.aws_instance.web: Refreshing state... [id=i-024af9374d0826cb7]

```

★ Key Interview Explanation Points

Why ECR?

Because requirement says: “container image should be stored in registry”.

Why ALB?

Because requirement says: “service should not be accessible from internet except via load balancer”.

Why user data?

Because requirement says: “use terraform workaround instead of CI/CD”.

So user_data automates docker pull + run.
